AWS SAA03 FULL COURSE CONTENT

🚀 What is EC2 (Amazon Elastic Compute Cloud)?

EC2 is an AWS service that provides **virtual servers in the cloud**.
You can use these servers to **run applications, websites, databases, or backend services**—without buying physical hardware.

☞ Think of EC2 as a **computer you rent from AWS**.

Why is it called *Elastic*?

Because you can:

- **Increase or decrease** server size anytime
- **Start, stop, or terminate** servers when needed
- Pay **only for what you use**

Key Features of EC2

1Virtual Server (Instance)

- EC2 server is called an **Instance**
- Runs OS like:
  - Linux
  - Windows

Launch a Linux EC2 Instance (Basic Flow)

1. EC2 → Launch Instance
2. Select AMI (Amazon Linux / Ubuntu)
3. Choose instance type (t2.micro – free tier)
4. Create/select key pair (.pem)
5. Configure Security Group (Allow SSH – port 22)
6. Launch instance

Connect to Linux EC2 (PuTTY)

- **PuTTYgen**: Convert `.pem` to `.ppk`

- **PuTTY**: SSH client

Steps:

- Hostname: `ec2-user@Public-IP`
- SSH → Auth → Load `.ppk` key

Basic Linux Commands (Must-Know)

```
sudo su
ls        pwd        clear
cd /      cd ..
rm file
yum update      (Amazon Linux)
apt-get update  (Ubuntu)
sudo vim file
```

Host Sample Website (Apache)

```
sudo yum install httpd -y
sudo systemctl start httpd
sudo systemctl enable httpd
cd /var/www/html
sudo vim index.html
```

Access via: `http://Public-IP`
(Security Group → Allow HTTP 80)

EC2 Storage – EBS

- **EBS** = Block storage attached to EC2
- Data persists even if instance stops

Increase Volume Size

- EC2 → Volumes → Modify Volume
- Resize inside OS using growpart

Snapshot & Backup

- **Snapshot** = Incremental backup of EBS
- Stored in AWS-managed S3
- Used for disaster recovery

AMI (Amazon Machine Image)

- Blueprint of EC2 instance
- Includes OS, apps, configurations
- Used to launch identical servers

EC2 Pricing Models

- **On-Demand**: Pay as you use
- **Reserved Instance**: 1–3 years, cheaper
- **Spot Instance**: Lowest cost, can be interrupted

Networking & Security

- **Security Group**: Stateful firewall
- **Public IP**: Changes on stop/start
- **Elastic IP**: Static IPv4 address

Windows EC2

- Use Windows AMI
- Allow RDP (3389)
- Decrypt password using key pair
- Connect via Remote Desktop

Add / Remove Disk (EBS)

- Create Volume → Attach to EC2
- Format and mount inside OS

Lifecycle Manager (DLM)

- Automates snapshot backups
- Tag-based scheduling

Savings Plans & Launch Template

- **Savings Plans**: Lower EC2 cost with commitment
- **Launch Template**: Reusable EC2 configuration

LOAD BALANCER & AUTO SCALING – SAA-C03 NOTES

Why Load Balancer & Auto Scaling?

To keep applications **highly available, scalable, and fault tolerant**.

- Load Balancer distributes traffic
- Auto Scaling adjusts EC2 count automatically

Milestone Architecture (One Website)

User → **ALB** → **Target Group** → **EC2 (Auto Scaling Group)**

- If traffic ↑ → EC2 added
- If EC2 fails → replaced automatically

Types of Load Balancer

1. Application Load Balancer (ALB)

- Layer 7 (HTTP/HTTPS)
- Path & host-based routing
- Works with **Target Groups**
- Used for web applications
-

2. Network Load Balancer (NLB)

- Layer 4 (TCP/UDP)
- Ultra-low latency
- Handles millions of requests
- Static IP support

Create an Application Load Balancer

Steps:

1. EC2 → Load Balancers → Create
2. Select **Application Load Balancer**
3. Internet-facing
4. Select VPC & subnets
5. Security Group → Allow HTTP/HTTPS
6. Attach Target Group

Load Balancer Details & Monitoring

- Health checks ensure only healthy EC2 get traffic
- Metrics in **CloudWatch**:
  - RequestCount
  - HealthyHostCount
  - TargetResponseTime

Target Group

- Logical group of EC2 instances
- Used by ALB/NLB to route traffic

Create Target Group:

- Target type: Instance
- Protocol: HTTP
- Port: 80
- Health check path: /index.html

AMI for Configuration

AMI captures:

- OS
- Web server (Apache/Nginx)
- App configuration

Create AMI: EC2 → Instance → Actions → Create Image

Used to launch identical EC2 in Auto Scaling

Launch Template

Reusable EC2 configuration:

- AMI
- Instance type
- Key pair
- Security group

Why? Auto Scaling uses Launch Template to create instances

Auto Scaling Group (ASG)

Automatically manages EC2 instances

Setup:

1. Choose Launch Template
2. Select VPC & subnets
3. Attach Load Balancer
4. Set Min / Desired / Max capacity

Example:

- Min: 1
- Desired: 2
- Max: 4

Auto Scaling Policies

Define **when to scale**

Types:

- Target Tracking (Most common)
- Step Scaling
- Scheduled Scaling

Example Policy:

- CPU > 70% → Add EC2
- CPU < 30% → Remove EC2

Attach ASG with Load Balancer

- During ASG creation
- Select Target Group
- ASG auto-registers instances

🌐 AWS Route 53 – DNS Service for Website Hosting

What is Route 53?

Amazon Route 53 is a **DNS (Domain Name System)** service that:

- Converts **domain names** (example.com) into **IP addresses**
- Routes user traffic to servers, load balancers, or AWS services
- Provides **high availability and health checking**

Hosted Zone

A **Hosted Zone** is a container that holds DNS records for your domain.

Types of Hosted Zones:

1. **Public Hosted Zone**
   - Used for websites accessible on the internet
   - Example: google.com
2. **Private Hosted Zone**
   - Used inside VPC (internal apps)
   - Not accessible from the internet

Create Hosted Zone (Custom Domain):

Steps:

1. Go to **Route 53 → Hosted Zones**
2. Click **Create Hosted Zone**
3. Enter domain name (example.com)
4. Choose **Public Hosted Zone**
5. Route 53 generates **Name Servers (NS)**
6. Update these NS in your domain registrar

Routing Policies

Routing policies decide **how traffic is routed**.

1 Simple Routing

- One record → one resource
- No health check
- Use case: single server website

2 Weighted Routing

- Split traffic by percentage
- Example: 70% to Server A, 30% to Server B
- Use case: A/B testing

3 Failover Routing

- Primary + Secondary resource
- Uses health check
- If primary fails → traffic goes to secondary
- Use case: disaster recovery

4 Latency-based Routing

- Routes users to the **lowest latency** region
- Example: India user → Mumbai region
- Use case: global applications

5 Geolocation Routing

- Routes based on **user country/continent**
- Example: India users → Indian server
- Use case: content localization

Record Types (DNS Records)

A Record

- Maps domain → IPv4 address
- Example: example.com → 13.234.XX.XX

Alias Record (AWS Special)

- Points domain to:
  - Load Balancer
  - CloudFront
  - S3 website
- No extra cost, auto-updates IP

MX Record

- Used for **email routing**
- Example: Gmail, Outlook mail servers
-

CNAME

- One domain → another domain
- Example: www → example.com

Health Checks

Health checks monitor resource health.

IP-based Health Check

- Checks if IP is reachable
- Used for EC2 or on-prem servers

URL-based Health Check

- Checks specific URL (HTTP/HTTPS)
- Example: `/health` endpoint

Steps:

1. Route 53 → Health Checks
2. Create Health Check
3. Choose IP or Domain
4. Set protocol (HTTP/HTTPS)
5. Attach with routing policy (Failover / Weighted)

How to Purchase Domain

Ways to buy domain:

1. **AWS Route 53 Domains**
2. External providers:
    - GoDaddy
    - Namecheap

If purchased outside AWS:

- Update **Name Servers** with Route 53 NS values

Traffic Policies (Intro)

Traffic Policy helps manage **complex routing logic** visually.

- Used for advanced routing
- Version controlled
- Reusable across hosted zones

Attach Traffic Policy:

1. Create Traffic Policy
2. Create Policy Record
3. Attach to Hosted Zone

How to Map Custom Domain with Server / Load Balancer

Domain → EC2

- Create **A Record**
- Point to EC2 public IP

Domain → Load Balancer

- Create **Alias Record**
- Select Load Balancer
- Recommended approach (auto-scaling safe)

Amazon S3 (Simple Storage Service)

Object-Based Storage

What is S3?

Amazon S3 is an **object storage service** used to store files like:

- Images, videos, PDFs
- Backups
- Logs
- Static website files

☞ Stores data as **Objects inside Buckets**

🎁 S3 Storage Classes

1 Standard

- Frequently accessed data
- High availability & durability
- Example: website images

2. RRS (Reduced Redundancy Storage) – *Old / Previous Gen*

- Lower durability
- **Not recommended now**

3.Infrequent Access (IA)

- Data accessed rarely
- Lower cost than Standard
- Example: backups

4.One Zone-IA

- Stored in **single AZ**
- Cheaper than IA
- Example: non-critical data

5.Glacier

- Archival storage
- Very low cost
- Retrieval takes minutes to hours
- Example: old records

How to Create an S3 Bucket

Steps:

1. Go to **S3 → Create Bucket**
2. Enter unique bucket name
3. Select region
4. Configure:
    - Versioning (optional)
    - Encryption (optional)
5. Create bucket

📂 Upload & Delete Objects

Upload:

- Open bucket → Upload → Select file → Upload

Delete:

- Select object → Delete

⚙ Bucket Properties

Versioning

- Keeps multiple versions of object
- Protects from accidental delete
- Enabled at bucket level

Logging

- Stores access logs
- Useful for auditing

Event Notification

- Trigger actions when object is uploaded/deleted
- Can send events to:
    - Lambda
    - SQS
    - SNS

🌍 Make Bucket/Object Public

Steps:

1. Disable **Block Public Access**
2. Add **Bucket Policy**
3. Set object permission to public

🌐 Static Website Hosting in S3

Steps:

1. Create bucket
2. Upload `index.html`
3. Enable **Static Website Hosting**
4. Make objects public
5. Access using S3 website URL
6. (Optional) Map custom domain via Route 53

🔁 Cross-Region Replication (CRR)

- Automatically copies objects to another region
- Requires:
  - Versioning enabled
  - IAM role

Use case:

- Disaster recovery
- Compliance

📜 Bucket Policy

- JSON-based permission
- Controls access to bucket & objects
- Used to allow:
  - Public access
  - Cross-account access

♻️ Lifecycle Policy

- Automatically move or delete objects
- Example:
  - After 30 days → IA
  - After 365 days → Glacier
  - After 5 years → Delete

🔒 Object Lock

- Prevents object deletion
- Used for compliance (WORM)
- Modes:
  - Governance
  - Compliance

🚀 Transfer Acceleration

- Faster uploads/downloads
- Uses CloudFront edge locations
- Enabled at bucket level

🌐 VPC (Virtual Private Cloud) – Overview

What is VPC?

VPC is a **private virtual network** inside AWS where you can launch your resources like **EC2, RDS, Load Balancer** securely.

👉 Think of VPC as **your own data center in the AWS cloud**.

Why VPC is Needed?

- Network isolation (security)
- Full control over IP addresses
- Control inbound & outbound traffic
- Design secure architectures

🏗 Create VPC Manually (Public & Private Subnet)

Manually creating VPC means:

- Define **CIDR block**
- Create **public subnet** (with Internet Gateway)
- Create **private subnet** (no direct internet)
- Configure **route tables** separately for public & private subnets

🖥 Launch Instances in VPC (Windows & Unix)

EC2 instances (Windows/Linux) are launched inside subnets:

- Windows → RDP access
- Linux/Unix → SSH access
  Security Groups control access.

🌍 Internet Connectivity of Public Subnet

Public subnet gets internet access when:

- Internet Gateway is attached
- Route table has `0.0.0.0/0 → IGW`
- Instance has public IP

🔐 Access Private Subnet from Public Subnet (Bastion / Jump Server)

A **Bastion Server** is an EC2 instance in public subnet used to:

- Securely connect to private instances
- No direct internet access to private subnet
- Improves security

Core VPC Components (Understanding)

VPC

Main network container

Subnet

Smaller network inside VPC

Route Table

Defines traffic direction

Internet Gateway (IGW)

Allows internet access

NAT Gateway

Allows private subnet to access internet (outbound only)

Egress Internet Gateway

Used for **IPv6 outbound-only internet access**

⍰ Test Lab

A test lab is a **practice setup** used to:

- Verify connectivity
- Test routing, security rules
- Understand real AWS behavior

📊 VPC Flow Logs

Captures network traffic metadata:

- Source IP
- Destination IP

- Accept / Reject
  Used for:
- Troubleshooting
- Security monitoring

⚙Setup VPC from Wizard Panel

AWS provides VPC Wizard **to quickly create:**

- **VPC**
- **Subnets**
- **IGW**
- **NAT Gateway**
  Best for beginners and fast setup.

🔗 Connectivity Between Different VPC

Different VPCs can communicate using:

- **VPC Peering**
- **Transit Gateway**
- **VPN**

🔒 VPN (Virtual Private Network)

Creates a **secure encrypted tunnel** between:

- On-premises network and AWS VPC
- Or between VPCs
  Uses IPsec.

🌍 Transit Gateway

A **central hub** to connect:

- Multiple VPCs
- VPNs
- Direct Connect
  Simplifies large network architectures.

RDS (Relational Database Service) – Overview

What is RDS?

Amazon RDS is a **managed relational database service** by AWS.

☞ AWS takes care of:

- Database setup
- Patching & upgrades
- Backups
- High availability

So you focus only on **data and applications**, not database maintenance.

Supported Database Engines

RDS supports:

- **MySQL**
- **PostgreSQL**
- **MariaDB**
- **Oracle**
- **SQL Server**

How to Create & Connect MySQL Database

Create:

1. Open **RDS → Create database**
2. Choose **MySQL**
3. Select instance size
4. Set username & password
5. Choose VPC & subnet group
6. Create database

Connect:

- From EC2 using:

```
mysql -h endpoint -u admin -p
```

Backup & Restore MySQL Database

Automatic Backup

- Enabled by default
- Point-in-time recovery
- Stored in S3 (managed by AWS)

Manual Backup

- Create **DB Snapshot**
- Used for restore anytime

Restore

- Restore snapshot → new RDS instance

Automatic Maintenance (Minor Patch Upgrades)

- AWS applies **minor DB patches automatically**
- Happens during **maintenance window**
- Improves security & stability
- No data loss

DynamoDB / NoSQL Database

DynamoDB is a **NoSQL key-value database**:

- Fully managed
- Serverless
- Extremely fast
- No fixed schema

Use case:

- Real-time apps
- Gaming
- IoT
- High-scale workloads

Read Replica in RDS

A **Read Replica** is a copy of primary DB used for **read operations**.

Benefits:

- Improves read performance
- Reduces load on main DB
- Used for analytics & reporting

⚠Read replica is **not for failover** (that is Multi-AZ)

⚡ ElastiCache Service

What is ElastiCache?

ElastiCache is an **in-memory caching service** used to:

- Reduce database load
- Improve application performance

ElastiCache Engines

Redis

- Supports:
  - Data persistence
  - Replication
  - Backup
- Used for:
  - Session storage
  - Leaderboards
-

Memcached

- Simple key-value store
- No persistence
- Faster but basic
- Used for caching objects

Example – Redis & Memcached (Test Use Case)

Redis Example

- App checks Redis for data
- If found → return fast
- If not → fetch from RDS and store in Redis

Memcached Example

- Frequently accessed website data
- Stored temporarily in memory
- Reduces DB queries

🔐 IAM (Identity and Access Management) – Overview

What is IAM?

IAM is an AWS service used to **control who can access AWS resources and what actions they can perform**.

☞ It helps you manage **users, permissions, and security** in AWS.

Why IAM is Important?

- Secure AWS account access
- Avoid using root account
- Follow **least privilege principle**
- Control access to services like EC2, S3, RDS

👤 How to Create IAM User

Steps:

1. IAM → Users → Create user
2. Enter username
3. Select access type:
    - Console access
    - Programmatic access
4. Attach permissions
5. Create user

Used for **individual people or applications**.

👥 How to Create IAM Group

Steps:

1. IAM → Groups → Create group
2. Attach policies
3. Add users to group

Group makes **permission management easier**.

🎭 How to Create IAM Role

Steps:

1. IAM → Roles → Create role
2. Choose trusted entity (EC2, Lambda, AWS Account)
3. Attach policies
4. Create role

Roles provide **temporary access without passwords**.

💻 AWS CLI Access (Programmatic & Console)

Programmatic Access

- Used for CLI/SDK
- Uses **Access Key & Secret Key**

Console Access

- Used for AWS Management Console
- Uses username & password

📜 What is IAM Policy?

A **policy** is a JSON document that defines:

- What actions are allowed or denied
- On which resources

Example:

```
Allow s3:PutObject on a bucket
```

🔗 How Policies Are Applied

- **User** → Direct permissions
- **Group** → All users inherit permissions
- **Role** → Assumed by AWS services or users

🔐 How to Enable MFA

Steps:

1. IAM → Users → Security credentials
2. Assign MFA device
3. Scan QR code
4. Enter OTP

Adds **extra security layer**.

Using IAM Policies (Managed & Custom)

AWS Managed Policies

- Created by AWS
- Ready to use
- Example: `AmazonS3ReadOnlyAccess`

Custom Policies

- Created by user
- Fine-grained control
- Written in JSON

How to Create Customized Account Link

Steps:

1. IAM → Dashboard
2. Customize **AWS Account Alias**
3. Login URL becomes:

https://your-alias.signin.aws.amazon.com

🔑 Password Policy Setup

Steps:

1. IAM → Account settings
2. Set:
    - Minimum length
    - Complexity
    - Expiry
    - Reuse prevention

Used for **organization security**.

📊 How to Fetch IAM Reports

IAM provides:

- **Credential Report**
  - Password age
  - Access key status
  - MFA status
- Used for **security audits**

Path:
IAM → Credential Report → Download

🌱 AWS Elastic Beanstalk – Overview

What is Elastic Beanstalk?

AWS Elastic Beanstalk is a **Platform as a Service (PaaS)** that lets you **deploy and manage applications easily** without worrying about infrastructure.

☞ You just upload your code, AWS handles:

- EC2
- Load Balancer
- Auto Scaling
- Monitoring

Supported Platforms

Elastic Beanstalk supports:

- **Java**
- **.NET**
- **Node.js**
- **Python**
- **PHP**
- **Ruby**
- **Go**

How Elastic Beanstalk Works

1. Upload application code
2. Choose platform (Node.js, Java, etc.)
3. Elastic Beanstalk automatically:
    - Creates EC2
    - Sets up Load Balancer
    - Configures Auto Scaling
4. Application becomes live

Benefits

- Easy deployment
- Automatic scaling
- Built-in monitoring
- No infra management

🌍 Amazon CloudFront – CDN Service

What is CloudFront?

Amazon CloudFront is a **Content Delivery Network (CDN)** that:

- Delivers content faster
- Reduces latency
- Improves website performance

☞ It serves content from **edge locations** close to users.

CloudFront Distribution

A **Distribution** is the CloudFront configuration that:

- Defines where content comes from
- Controls caching & security
- Exposes a CloudFront URL

Types:

- **Web Distribution** – Websites, APIs
- **RTMP** (legacy, rarely used)

Origin Source

Origin is the source of original content.

Common origins:

- **S3 bucket** (static websites)
- **EC2 / Load Balancer**
- **Elastic Beanstalk**
- **Custom HTTP server**

Edge Locations (Reduce Latency)

- Global data centers
- Cache content closer to users
- Less distance = faster response

Example:
User in India → Mumbai Edge → Faster load

🏗 AWS CloudFormation – Overview

What is CloudFormation?

CloudFormation is an **Infrastructure as Code (IaC)** service.

☞ It lets you **create and manage AWS resources using code**, not manual clicks.

How CloudFormation Works

- You write a **template** (YAML/JSON)
- CloudFormation creates resources automatically
- All resources are managed as a **stack**

Benefits

- Automation
- Consistency
- Easy rollback
- Version control

📢 SNS (Simple Notification Service)

What is SNS?

SNS is a **messaging service** used to send **instant notifications** to many subscribers at the same time.

☞ Best for **transactional messages** and **real-time alerts**.

Key Features

- Pub/Sub (Publish–Subscribe) model
- Near real-time delivery
- Highly scalable

Message Types

- **SMS notifications**
- Email
- HTTP/HTTPS
- Lambda
- SQS

Topic & Subscription (Must)

- **Topic** → Central channel where messages are published
- **Subscription** → Endpoint that receives messages

⚠ **Subscription is mandatory**
No subscriber = no message delivery

Common Use Cases

- OTP messages
- System alerts
- Billing notifications
- Application events

📧 SES (Simple Email Service)

What is SES?

SES is an email service used to send **emails at scale**.

☞ Best for **bulk, promotional, and marketing emails**.

Key Features

- Low-cost email sending
- High deliverability
- Supports HTML & text emails

Email Types

- **Bulk marketing emails**
- Promotional messages
- Transactional emails (also possible)

Subscription (Not Mandatory)

- SES sends emails **directly to email addresses**
- No topic-subscription model like SNS
- Recipient list is managed by the application

SES Requirements

- Verify email/domain
- Sandbox → Production access (for high volume)

Common Use Cases

- Newsletters
- Offers & promotions
- Account emails
- Marketing campaigns

📜 AWS CloudTrail – Overview

What is CloudTrail?

AWS CloudTrail is a service that logs every action (event) performed in your AWS account.

☞ It records who did what, when, and from where.

What Does CloudTrail Log?

- AWS Console actions
- AWS CLI commands
- API calls from SDKs
- Changes to AWS resources

Examples:

- EC2 started/stopped
- S3 bucket created/deleted
- IAM policy attached

Key Features

- Logs are stored in **S3**
- Helps in **security auditing**
- Useful for **troubleshooting**
- Required for **compliance**

Event Types

- **Management Events** – Resource changes (default)
- **Data Events** – S3 object access, Lambda invoke
- **Insight Events** – Unusual activity detection

📬 SQS (Simple Queue Service) – Overview

What is SQS?

SQS is a **fully managed message queue service** used to:

- Decouple applications
- Handle background processing
- Improve reliability

☞ Messages are stored in a queue until processed.

Why Use SQS?

- Prevents system overload
- Handles traffic spikes
- Ensures message delivery

SQS Queue Types

1.Standard Queue

- High throughput
- Messages delivered **at least once**
- Order not guaranteed

2.FIFO Queue

- Messages delivered **exactly once**
- Order preserved
- Lower throughput

Key Features

- Message retention (up to 14 days)
- Visibility timeout
- Dead Letter Queue (DLQ)

Common Use Cases

- Order processing
- Background jobs
- Email sending
- Image processing

🔄 SWF (Simple Workflow Service)

What is SWF?

AWS SWF is a service used to **coordinate and manage workflows** that have **multiple steps** and may run for a long time.

☞ It helps manage **task order, retries, and state tracking**.

Key Concepts

- **Workflow** → Complete business process
- **Activity Tasks** → Individual steps
- **Decider** → Controls task flow and decisions
- **Workers** → Perform tasks

Features

- Reliable task execution
- Handles retries and failures
- Tracks task progress

Use Cases

- Order processing
- Video encoding pipelines
- Payment workflows

🌊 Amazon Kinesis – Overview

What is Kinesis?

Amazon Kinesis is a **real-time data streaming service** used to **collect, process, and analyze streaming data**.

☞ Works with data that comes continuously.

Kinesis Services

1.Kinesis Data Streams

- Real-time data ingestion
- Used for custom stream processing

2.Kinesis Data Firehose

- Load streaming data into:
  - S3
  - Redshift
  - OpenSearch
- Fully managed, no coding

3 Kinesis Data Analytics

- Analyze streaming data using SQL

Key Features

- Real-time processing
- Scalable
- Low latency

Use Cases

- Log analytics
- Clickstream analysis
- IoT data processing
- Real-time monitoring

🔐 KMS (Key Management Service)

What is KMS?

AWS KMS is a service used to **create, manage, and control encryption keys**.

☞ It helps protect your data using **encryption**.

Why KMS is Used?

- Secure sensitive data
- Control who can use encryption keys
- Meet security & compliance needs

Key Concepts

- **Customer Managed Keys (CMK)** – Keys you control
- **AWS Managed Keys** – Keys created & managed by AWS
- **Encryption & Decryption** – Data protection

Key Features

- Centralized key management
- Integrated with IAM
- Automatic key rotation
- Audit via CloudTrail

⚡ AWS Lambda – Overview

What is Lambda?

AWS Lambda is a **serverless compute service** that runs code **without managing servers**.

☞ You upload code, AWS runs it automatically.

How Lambda Works

- Triggered by events
- Executes code
- Scales automatically
- You pay only for execution time

Supported Languages

- Python
- Node.js
- Java
- Go

Key Benefits

- No server management
- Auto scaling
- Cost effective
- Highly available

Common Use Cases

- Backend APIs
- File processing
- Real-time data handling
- Automation tasks

How AWS Pricing Works?

💰 AWS Pricing – Overview

AWS follows a **pay-as-you-go model**.

👉 You pay **only for what you use**, no upfront cost (mostly).

AWS Pricing Factors

Cost depends on:

- Service used (EC2, S3, RDS, etc.)
- Usage time (hours/seconds)
- Storage size
- Data transfer
- Region

Common Pricing Models

- **On-Demand** – Pay as you go
- **Reserved** – Lower cost for long-term usage
- **Spot** – Cheapest (unused AWS capacity)
- **Free Tier** – Limited free usage for new users

▯ AWS Pricing Calculator

What is AWS Pricing Calculator?

AWS Pricing Calculator helps you **estimate monthly AWS cost** before using services.

☞ Very useful to avoid unexpected bills.

What You Can Do with Calculator

- Estimate EC2, S3, RDS, Lambda costs
- Compare instance types
- Plan budget for projects
- Export cost estimates

Simple Steps

1. Open AWS Pricing Calculator
2. Select services
3. Enter usage details
4. View estimated monthly cost

❄ AWS Snowball – Overview

What is AWS Snowball?

AWS Snowball is a **physical data transfer device** used to move **large amounts of data** into or out of AWS.

☞ Used when **internet transfer is slow, costly, or not possible**.

Why Snowball is Needed?

- Very large data (TBs / PBs)
- Faster than internet for bulk data
- Secure data transfer

Types of Snowball Devices

- **Snowball Edge Storage Optimized**
- **Snowball Edge Compute Optimized**

Key Features

- Encrypted data transfer
- Tamper-resistant device
- Integrated with S3 & EC2
- Offline data migration

Use Cases

- Data center migration
- Disaster recovery
- Backup transfer
- Media archives

📜 AWS CloudTrail – Overview

What is CloudTrail?

AWS CloudTrail is a service that **records every activity (event)** happening in your AWS account.

☞ It tells you **who did what, when, and from where**.

What Does CloudTrail Track?

- Console actions (login, create/delete resources)
- CLI & API calls
- SDK operations
- Changes in AWS services (EC2, S3, IAM, RDS)

Key Features

- Logs stored in **S3**
- Integrated with **CloudWatch Logs**
- Supports **security auditing & compliance**
- Helps in troubleshooting issues

CloudTrail Event Types

- **Management Events** – Resource operations (default)
- **Data Events** – S3 object-level & Lambda events
- **Insight Events** – Detects unusual activity

Use Cases

- Security monitoring
- Compliance reporting
- Root cause analysis
- Change tracking

AWS Storage Gateway – Overview

What is Storage Gateway?

AWS Storage Gateway is a **hybrid cloud service** that connects **on-premises storage** with **AWS cloud storage**.

☞ It allows local applications to use AWS storage **as if it were local storage**.

Types of Storage Gateway

1.File Gateway

- Stores files as objects in **S3**
- Supports NFS & SMB
- Use case: File shares, backups

2.Volume Gateway

- Block storage for on-prem servers
- Two modes:
  - **Cached Volumes** – Most data in S3
  - **Stored Volumes** – Data stored locally, backed up to AWS

3.Tape Gateway

- Virtual tape library (VTL)
- Replaces physical tape backup
- Uses **S3 Glacier / Deep Archive**

Common Use Cases

- Backup & archiving
- Disaster recovery
- File sharing
- Data migration

🔗 AWS Direct Connect – Overview

What is Direct Connect?

AWS Direct Connect (DX) is a **dedicated private network connection** between your **on-premises data center** and **AWS**.

☞ It **does not use the public internet**.

Why Use Direct Connect?

- Consistent network performance
- Lower latency
- Higher bandwidth
- More secure than internet VPN

How Direct Connect Works

1. Physical connection from your data center to AWS DX location
2. Virtual Interface (VIF) created
3. Traffic flows privately into AWS

Types of Virtual Interfaces (VIF)

Private VIF

- Connects to **VPC**
- Access EC2, RDS (private IP)

Public VIF

- Access AWS public services
- Example: S3, DynamoDB

Transit VIF

- Used with **Transit Gateway**
- Connect multiple VPCs

Common Use Cases

- Hybrid cloud setup
- Large data transfer
- Enterprise workloads
- Stable network connectivity

⚙Best Practices for High Availability (HA)

1.Use Redundant Connections

- Create **multiple DX connections**
- Use **different DX locations**

2.Combine DX + VPN

- Use **Site-to-Site VPN** as backup
- Automatic failover if DX fails

3.Multi-AZ Architecture

- Distribute workloads across AZs
- Avoid single point of failure

4.Health Monitoring

- Use CloudWatch & VPC Flow Logs
- Monitor latency & packet loss